El teorema del eje intermedio

(efecto Dzhanibekov o "de la raqueta de tenis"): la física, las matemáticas y el código

AM1 – Grupo 1 (MUSE, UPM)

24 de octubre de 2026

Resumen

Este documento explica el teorema del eje intermedio y el código de la carpeta `Problema de la raqueta`. Primero se cuenta **qué se observa** y su significado físico, con analogías. Después vienen **las ecuaciones** y **por qué** el eje intermedio es inestable. Por último se explica **qué hace cada archivo de Python**, línea a línea, y si la simulación es fiable, conectándolo con los métodos numéricos de los Milestones 1–3. Las figuras y los números salen de ejecutar tu código. En algunos casos se compara con una solución de referencia muy precisa calculada aparte.

Índice

1. El fenómeno **2**
- 1.1. Qué se observa . . . 2
- 1.2. Los tres ejes y los momentos de inercia . . . 2
- 1.3. El enunciado del teorema . . . 3

2. La física: las ecuaciones de Euler **4**
- 2.1. Velocidad angular y momento angular . . . 4
- 2.2. Si $\mathbf{L}$ no cambia, ¿por qué cambia $\boldsymbol{\omega}$? . . . 4

3. Dos cantidades que nunca cambian **5**
- 3.1. Energía y momento angular . . . 5
- 3.2. La imagen geométrica: una esfera y un elipsoide . . . 5

4. ¿Por qué el eje intermedio es inestable? Análisis lineal **7**
- 4.1. La cuenta . . . 7
- 4.2. Los números de tu caso: $I = (1, 2, 3)$, $\Omega = 1$ . . . 7
- 4.3. ¿Cuánto tarda en voltearse? Muy poco, aunque el giro sea casi perfecto . . . 8

5. Después del volteo: por qué vuelve **9**

6. El código, archivo por archivo **10**
- 6.1. `Problema_raqueta_V1.py`: la primera simulación . . . 10
- 6.2. `Problema_raqueta_V2.py`: más tiempo, varios volteos . . . 12
- 6.3. `Problema_raqueta_V3.py`: el experimento de control . . . 12
- 6.4. `Problema_raqueta_V4.py`: ver girar el cuerpo (cuaterniones) . . . 12
- 6.5. `Problema_raqueta_V5.py`: el laboratorio interactivo . . . 17

7. ¿Es fiable la simulación? (conexión con los Milestones) **22**
- 7.1. Euler explícito siempre añade energía . . . 22
- 7.2. Con los métodos del Milestone 3 . . . 23
- 7.3. Recomendación . . . 24

8. Resumen **24**

1. El fenómeno

1.1. Qué se observa

Coge un móvil, un libro o una raqueta de tenis y lánzalo al aire haciéndolo girar. Un objeto así tiene tres ejes naturales de giro, y el resultado depende de cuál elijas:

- Alrededor del eje **largo** (en la raqueta, el del mango) gira limpio y estable.
- Alrededor del eje **perpendicular a la cara** también gira limpio y estable.
- Alrededor del **tercer** eje, el que está en el plano de la cara y es perpendicular al mango, pasa algo sorprendente. Además de girar, el objeto **da media vuelta sobre sí mismo** (un "volteo"). Lo hace aunque lo hayas lanzado con muchísimo cuidado y sin que nada lo toque en el aire.

En 1985 el cosmonauta Vladímir Dzhanibekov lo vio a bordo de la estación espacial Salyut 7. Una tuerca de mariposa que salía girando de un tornillo se volteaba periódicamente mientras flotaba. Por eso se llama **efecto Dzhanibekov**. El resultado matemático que lo explica es el **teorema del eje intermedio** o **de la raqueta de tenis**.

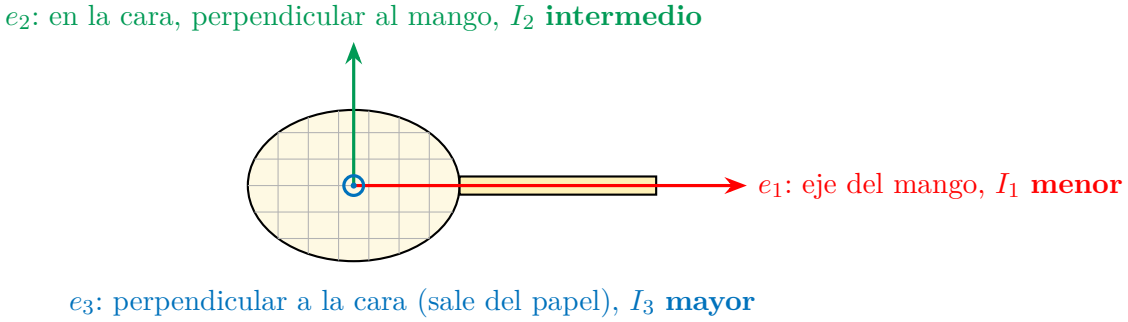
1.2. Los tres ejes y los momentos de inercia

Igual que la *masa* mide cuánto le cuesta a un cuerpo acelerar, el **momento de inercia** I mide **cuánto le cuesta girar** alrededor de un eje. Depende de cómo de lejos del eje está repartida la masa.

Analogía: la patinadora

Una patinadora que gira con los brazos abiertos tiene la masa lejos del eje, así que su momento de inercia es grande y gira despacio. Si recoge los brazos, I disminuye y gira mucho más deprisa. Cuanto más lejos del eje está la masa, más "pereza para girar" tiene el cuerpo.

Todo cuerpo rígido tiene tres **ejes principales**, perpendiculares entre sí, con tres momentos de inercia $I_1 < I_2 < I_3$:



- e_1 (mango): la masa está muy cerca de ese eje, así que I_1 es el menor.
- e_3 (perpendicular a la cara): toda la masa de la cara está lejos de él, así que I_3 es el mayor.
- e_2: queda en medio, con I_2 intermedio. **Este es el eje problemático.**

Para un bloque homogéneo de lados a, b, c (a lo largo de e_1, e_2, e_3) y masa m:

$$I_1 = \tfrac{m}{12}(b^2 + c^2), \qquad I_2 = \tfrac{m}{12}(a^2 + c^2), \qquad I_3 = \tfrac{m}{12}(a^2 + b^2).$$

Idea clave: tu código simula una hoja de papel A4 rígida

El código usa $I_1 = 1$, $I_2 = 2$, $I_3 = 3$. Fíjate en que $I_1 + I_2 = I_3$. Eso solo ocurre en un cuerpo *plano* (grosor $c = 0$), porque entonces $I_1 = \frac{m}{12}b^2$, $I_2 = \frac{m}{12}a^2$ e $I_3 = I_1 + I_2$. Además, $I_2/I_1 = a^2/b^2 = 2$ implica $a/b = \sqrt{2}$, que es justo la proporción de una **hoja A4** ($297/210 = 1.414$). Así que los números de tu código corresponden a una lámina fina con forma de A4, que gira de forma estable alrededor de su lado largo y de su normal, y se voltea alrededor de su lado corto.
Esto también enseña una restricción física: en un cuerpo real ningún momento puede superar la suma de los otros dos ($I_3 \leq I_1 + I_2$). El trío $(1, 2, 3)$ está justo en el límite.

1.3. El enunciado del teorema

Idea clave

Teorema del eje intermedio. Un cuerpo rígido libre, sin fuerzas que lo hagan girar, con tres momentos de inercia distintos $I_1 < I_2 < I_3$:

- gira de forma **estable** alrededor del eje de **menor** inercia (e_1) y del de **mayor** inercia (e_3);
- gira de forma **inestable** alrededor del eje **intermedio** (e_2).

"Estable" quiere decir que, si el giro empieza un poquito torcido, se queda un poquito torcido. "Inestable" quiere decir que ese poquito **crece** hasta producir el volteo.

2. La física: las ecuaciones de Euler

2.1. Velocidad angular y momento angular

- La **velocidad angular** $\boldsymbol{\omega} = (\omega_1, \omega_2, \omega_3)$ dice con qué rapidez gira el cuerpo alrededor de cada uno de sus tres ejes. Es lo que dibujan las gráficas del código: "omega 1", "omega 2", "omega 3".
- El **momento angular** es la "cantidad de giro", el equivalente para rotaciones del momento lineal $p = mv$:

$$\mathbf{L} = (I_1\omega_1,\ I_2\omega_2,\ I_3\omega_3).$$

Si nada empuja al cuerpo para hacerlo girar (no hay *par* externo), **L se conserva**: apunta siempre hacia el mismo sitio del espacio, con el mismo tamaño. Es la versión giratoria de "un objeto sin fuerzas sigue en línea recta".

2.2. Si L no cambia, ¿por qué cambia ω?

Porque las componentes $\omega_1, \omega_2, \omega_3$ se miden **respecto a los ejes del propio cuerpo**, y esos ejes giran con él.

Analogía: el tiovivo y la farola

Montado en un tiovivo, miras una farola que está quieta en la calle. Desde tu asiento, la farola parece dar vueltas a tu alrededor. Ha cambiado tu punto de vista, no la farola. Con el cuerpo rígido pasa lo mismo: **L** está quieto en el espacio, pero visto desde los ejes del cuerpo, que giran, parece moverse.

La regla que traduce "quieto en el espacio" a "visto desde algo que gira" es

$$\underbrace{\frac{d\mathbf{L}}{dt}}_{\text{cambio visto desde el cuerpo}} + \boldsymbol{\omega} \times \mathbf{L} \ = \ \mathbf{0}.$$

Escribiéndola componente a componente salen las **ecuaciones de Euler del cuerpo rígido**:

$$\boxed{\dot{\omega}_1 = \frac{I_2 - I_3}{I_1}\,\omega_2\omega_3, \qquad \dot{\omega}_2 = \frac{I_3 - I_1}{I_2}\,\omega_3\omega_1, \qquad \dot{\omega}_3 = \frac{I_1 - I_2}{I_3}\,\omega_1\omega_2} \tag{1}$$

Cómo leerlas:

- Cada velocidad de giro cambia según el **producto de las otras dos**. Si el cuerpo solo gira alrededor de un eje (las otras dos son cero), nada cambia: es un giro de equilibrio.
- Son **no lineales**, porque aparecen productos $\omega_i\omega_j$. Es la gran diferencia con el oscilador armónico de los Milestones, donde **F** era una matriz por el estado.
- El signo de cada coeficiente importa. Con $I_1 < I_2 < I_3$: $\frac{I_2-I_3}{I_1} < 0$, $\frac{I_3-I_1}{I_2} > 0$ y $\frac{I_1-I_2}{I_3} < 0$.

Conexión con los Milestones

Es un problema de Cauchy igual que los de los Milestones:

$$\frac{d\mathbf{U}}{dt} = \mathbf{F}(\mathbf{U}), \qquad \mathbf{U} = \begin{pmatrix}\omega_1\\ \omega_2\\ \omega_3\end{pmatrix}, \qquad \mathbf{F}(\mathbf{U}) = \begin{pmatrix}\frac{I_2-I_3}{I_1}\omega_2\omega_3\\ \frac{I_3-I_1}{I_2}\omega_3\omega_1\\ \frac{I_1-I_2}{I_3}\omega_1\omega_2\end{pmatrix}, \qquad \mathbf{U}_0 = (\varepsilon,\ \Omega,\ \varepsilon).$$

Ahora el "estado" no es posición y velocidad, sino **las tres velocidades de giro**. La función `F(U)` de tu código es exactamente esta. La orientación del cuerpo (hacia dónde apunta) no está en **U**; la añade el archivo V4 con cuaterniones.

3. Dos cantidades que nunca cambian

3.1. Energía y momento angular

Sin rozamiento ni pares externos se conservan:

$$2T = I_1\omega_1^2 + I_2\omega_2^2 + I_3\omega_3^2 \quad \text{(el doble de la energía de rotación)},$$

$$|\mathbf{L}|^2 = I_1^2\omega_1^2 + I_2^2\omega_2^2 + I_3^2\omega_3^2 \quad \text{(el tamaño del momento angular, al cuadrado)}.$$

Se comprueba derivando y usando (1). Para la energía:

$$\frac{d(2T)}{dt} = 2\sum_i I_i\omega_i\dot{\omega}_i = 2\,\omega_1\omega_2\omega_3\big[(I_2 - I_3) + (I_3 - I_1) + (I_1 - I_2)\big] = 0.$$

Con $|\mathbf{L}|^2$ el corchete es $I_1(I_2 - I_3) + I_2(I_3 - I_1) + I_3(I_1 - I_2)$, que también suma cero.

3.2. La imagen geométrica: una esfera y un elipsoide

Pensemos en el vector $\mathbf{L}$ visto desde el cuerpo, $(L_1, L_2, L_3) = (I_1\omega_1, I_2\omega_2, I_3\omega_3)$. Las dos leyes de conservación dicen:

- $L_1^2 + L_2^2 + L_3^2 = |\mathbf{L}|^2$: la punta de $\mathbf{L}$ se mueve sobre una **esfera**.
- $\frac{L_1^2}{I_1} + \frac{L_2^2}{I_2} + \frac{L_3^2}{I_3} = 2T$: y además sobre un **elipsoide**.

La punta de $\mathbf{L}$ tiene que estar en los dos a la vez, así que recorre su **intersección**. Esa intersección son curvas cerradas sobre la esfera llamadas **polhodias** (Fig. 1).

Trayectorias del momento angular visto desde el cuerpo. En rojo, la separatriz que pasa por el eje 2

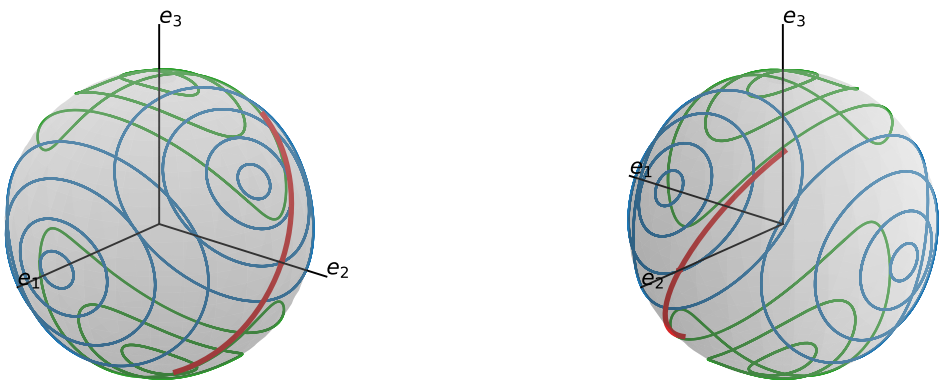


Figura 1: El camino que sigue el momento angular visto desde el cuerpo, en dos vistas de la misma esfera. Azul: órbitas alrededor de e_1 (menor inercia). Verde: órbitas alrededor de e_3 (mayor inercia). Las dos familias son círculos pequeños y cerrados alrededor de su eje: giro estable. Rojo: la **separatriz**, la frontera entre las dos familias, que pasa justo por $\pm e_2$. Alrededor del eje intermedio no hay círculos pequeños, sino un cruce en forma de X.

Analogía: el mapa de una montaña

Para un tamaño de **L** fijo, la energía depende del punto de la esfera. Imagina la esfera como un planeta con relieve donde la altura es la energía:

- $\pm e_1$ (menor inercia) son las **cimas**, de máxima energía. Las curvas de nivel alrededor de una cima son circulitos.
- $\pm e_3$ (mayor inercia) son los **fondos de valle**, de mínima energía. Alrededor también hay circulitos.
- $\pm e_2$ (intermedio) son **puertos de montaña**, o sillas de montar: suben en una dirección y bajan en otra. Las curvas de nivel se cruzan en X.

Como la energía se conserva, el punto se mueve a lo largo de una curva de nivel. Cerca de una cima o un valle da vueltecitas sin alejarse, así que es estable. Cerca de un puerto, la curva de nivel que pasa por él se aleja muchísimo y lleva al otro lado de la esfera: ese viaje es **el volteo**.

Idea clave

Las curvas son **cerradas**, así que el movimiento **se repite**: el cuerpo se voltea, vuelve, se voltea otra vez… con un periodo fijo. No es caótico ni aleatorio. Es un vaivén perfectamente regular, solo que muy sensible al punto de partida.

4. ¿Por qué el eje intermedio es inestable? Análisis lineal

4.1. La cuenta

Supón que el cuerpo gira casi del todo alrededor de e_2: $\omega_2 \approx \Omega$ (grande), con ω_1 y ω_3 muy pequeños (del tamaño de ε). En (1), el término $\omega_3\omega_1$ es un producto de dos cosas pequeñas, y por tanto *pequeñísimo*. Así que $\dot{\omega}_2 \approx 0$ y ω_2 se queda en Ω. Las otras dos ecuaciones quedan:

$$\dot{\omega}_1 = \underbrace{\frac{I_2 - I_3}{I_1}\Omega}_{a}\,\omega_3, \qquad \dot{\omega}_3 = \underbrace{\frac{I_1 - I_2}{I_3}\Omega}_{b}\,\omega_1.$$

Derivando la primera y sustituyendo la segunda:

$$\ddot{\omega}_1 = a\,\dot{\omega}_3 = ab\,\omega_1, \qquad ab = \frac{(I_3 - I_2)(I_2 - I_1)}{I_1 I_3}\,\Omega^2.$$

Todo depende del **signo** de ab:

- $ab < 0$: es $\ddot{\omega}_1 = -(\text{algo})\,\omega_1$, **el oscilador armónico** de los Milestones. La perturbación oscila sin crecer, así que el giro es estable.
- $ab > 0$: es $\ddot{\omega}_1 = +(\text{algo})\,\omega_1$, cuya solución contiene $e^{\sigma t}$ con $\sigma = \sqrt{ab}$. La perturbación **crece exponencialmente**, así que el giro es inestable.

Para e_2 los dos factores, $(I_3 - I_2)$ e $(I_2 - I_1)$, son positivos, luego $ab > 0$ y el giro es **inestable**. Repitiendo la cuenta para los otros dos ejes sale una regla muy limpia: *el producto es positivo justo cuando el eje de giro tiene inercia intermedia.*

Analogía: el lápiz

Un lápiz colgado de un hilo (como un péndulo), si lo empujas, oscila y vuelve: eso son los ejes e_1 y e_3. Un lápiz en equilibrio sobre la punta, si lo empujas lo más mínimo, se cae cada vez más deprisa: eso es el eje e_2. Y como el empujón inicial nunca es exactamente cero (un soplo de aire, un temblor de la mano), el lápiz en la punta *siempre* acaba cayendo.

4.2. Los números de tu caso: $I = (1, 2, 3)$, $\Omega = 1$

Giro alrededor de	ab	Comportamiento	Predicción
e_1 ($I_1 = 1$, menor)	$-1/3$	oscila	periodo $2\pi/\sqrt{1/3} = 10.88$
e_2 ($I_2 = 2$, intermedio)	$+1/3$	**crece**	$\sigma = 0.577$: $\times 10$ cada $\ln 10/\sigma = 4.0$
e_3 ($I_3 = 3$, mayor)	-1	oscila	periodo $2\pi = 6.28$

Lo que dice la simulación

Las tres predicciones se cumplen al medir sobre las simulaciones:

- Eje e_1 (V3): la perturbación oscila con periodo 10.88 y nunca supera 0.02.
- Eje e_3 (simulación extra): periodo 6.283.
- Eje e_2: la perturbación crece exactamente con pendiente $e^{0.577t}$ hasta casi el primer volteo (Fig. 2).

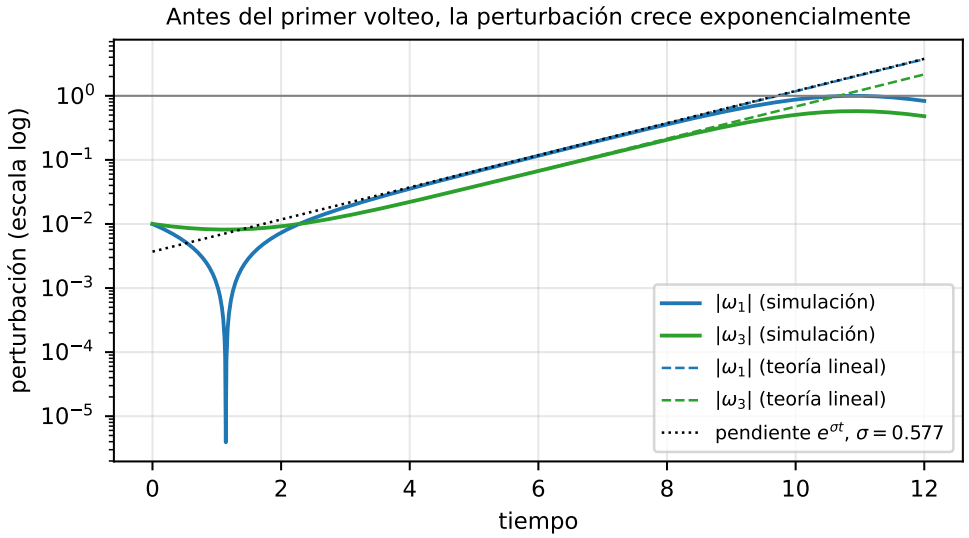


Figura 2: Escala logarítmica: una recta significa crecimiento exponencial. Las perturbaciones ω_1 y ω_3 (línea continua) siguen la teoría lineal (discontinua) hasta $t \approx 8$. Cuando llegan a tamaño ~ 1 deja de valer la aproximación "pequeño × pequeño ≈ 0", y empieza el volteo. (El pico hacia abajo de ω_1 en $t \approx 1.1$ es que cambia de signo y pasa por cero.)

Conexión con los Milestones: autovalores y estabilidad

En los Milestones vimos que la estabilidad se lee en los autovalores λ de la jacobiana $\partial\mathbf{F}/\partial\mathbf{U}$. Aquí, en el giro puro alrededor de cada eje:

- Ejes e_1 y e_3: $\lambda = \pm i\,\omega$, imaginarios puros, como el oscilador armónico. La física es estable y la solución exacta solo oscila.
- Eje e_2: $\lambda = \pm\sigma$, reales, y uno de ellos **positivo**. Es el caso "$\mathrm{Re}\,\lambda > 0$" que comentamos: el sistema es *físicamente* inestable, y lo que se le pide al método numérico no es que amortigüe, sino que crezca al ritmo correcto.

4.3. ¿Cuánto tarda en voltearse? Muy poco, aunque el giro sea casi perfecto

Si la perturbación empieza en ε y crece como $\varepsilon\, e^{\sigma t}$, se hace del tamaño del giro (~ 1) cuando

$$t_{\text{volteo}} \approx \frac{1}{\sigma} \ln \frac{1}{\varepsilon} + \text{constante}.$$

El **logaritmo** es la clave: dividir la perturbación entre 10 solo retrasa el volteo $\ln 10/\sigma = 4.0$ unidades de tiempo.

Lo que dice la simulación

Primer volteo (cuando ω_2 cambia de signo), con solución de referencia:

ε	0.1	0.01	0.001	0.0001	10^{-6}
t del primer volteo	6.88	10.92	14.91	18.89	26.87
salto respecto al anterior	–	4.04	3.99	3.98	7.98 (dos décadas)

Cada factor 10 añade casi exactamente 4.0, como predice la teoría. Incluso con una perturbación de una millonésima, el cuerpo se voltea antes de $t = 27$. Por eso el efecto es tan fácil de ver en la vida real: es imposible lanzar algo con un giro *perfecto*.

5. Después del volteo: por qué vuelve

La teoría lineal dice que la perturbación crece sin parar, pero eso solo vale mientras sea pequeña. La conservación de la energía y de $|\mathbf{L}|$ obliga al punto a seguir su curva cerrada sobre la esfera (Fig. 1). Así que ω_2 pasa de $+\Omega$ a $-\Omega$ (el cuerpo queda "boca abajo"), aguanta un rato así, y luego vuelve.

Analogía: el columpio que casi da la vuelta completa

Un columpio impulsado con la energía justa para llegar arriba del todo se queda **muchísimo tiempo** casi parado en lo alto, porque allí va lentísimo. Luego cae y pasa rápido por abajo. El cuerpo que gira alrededor de e_2 hace lo mismo: pasa largos ratos girando casi alrededor de $\pm e_2$, que es su "punto alto" inestable, y entre medias da volteos rápidos. Por eso las gráficas de ω_2 tienen **mesetas largas** en ± 1 unidas por **transiciones bruscas**.

El periodo exacto. La solución exacta de (1) se escribe con *funciones elípticas de Jacobi*, unas parientas del seno y el coseno. No hace falta conocerlas a fondo; basta saber que el periodo es

$$\tau = \frac{4K(k)}{\lambda}, \qquad \lambda^2 = \frac{(I_3 - I_2)(|\mathbf{L}|^2 - 2TI_1)}{I_1 I_2 I_3}, \qquad k^2 = \frac{(I_2 - I_1)(2TI_3 - |\mathbf{L}|^2)}{(I_3 - I_2)(|\mathbf{L}|^2 - 2TI_1)},$$

donde $K(k)$ es una integral clásica que `scipy.special.ellipk` calcula directamente. La fórmula vale si $|\mathbf{L}|^2 > 2TI_2$, que es tu caso; si no, se intercambian I_1 e I_3. Cuando el giro inicial se acerca al eje intermedio, $k \to 1$ y $K(k) \to \infty$: el periodo se hace infinito. Es la versión exacta de "un giro perfecto no se voltearía nunca".

Lo que dice la simulación

Para $\mathbf{U}_0 = (0.01, 1, 0.01)$, la fórmula da $\tau = 39.106$ y la simulación de referencia 39.106, que coinciden en las cinco cifras. ω_2 cambia de signo cada $\tau/2 = 19.55$, en $t = 10.92,\ 30.47,\ 50.02,\ 69.58,\ 89.13$.

6. El código, archivo por archivo

La carpeta contiene cinco versiones que van creciendo, y un `r.py` vacío (solo tiene un salto de línea).

Archivo	Qué hace	Novedad
`Problema_raqueta_V1.py`	giro casi en e_2, hasta $t = 20$	ecuaciones de Euler con Euler explícito
`Problema_raqueta_V2.py`	giro casi en e_2, hasta $t = 100$	se ven varios volteos
`Problema_raqueta_V3.py`	giro casi en e_1, hasta $t = 100$	comprobar que e_1 es estable
`Problema_raqueta_V4.py`	animación 3D del cuerpo	orientación con cuaterniones
`Problema_raqueta_V5.py`	ventana con controles	cambiar I y ω_0 y re-simular

6.1. `Problema_raqueta_V1.py`: la primera simulación

```python
from numpy import array, zeros
import matplotlib.pyplot as plt


# Momentos principales de inercia
I1 = 1.0
I2 = 2.0
I3 = 3.0


# Paso temporal
Dt = 0.001

# Número de pasos
N = 20000

# Número de variables del estado
Nv = 3


def F(U):
    w1 = U[0]
    w2 = U[1]
    w3 = U[2]

    dw1 = ((I2 - I3) / I1) * w2 * w3
    dw2 = ((I3 - I1) / I2) * w3 * w1
    dw3 = ((I1 - I2) / I3) * w1 * w2

    return array([dw1, dw2, dw3])


# Tabla donde guardamos toda la simulación
U = zeros((N + 1, Nv))

# Condición inicial:
# gira principalmente alrededor del eje intermedio
U[0, :] = array([0.01, 1.0, 0.01])


# Método de Euler
for n in range(0, N):
    U[n + 1, :] = U[n, :] + Dt * F(U[n, :])


# Vector de tiempos
t = zeros(N + 1)

```

```
49 for n in range(0, N):
50     t[n + 1] = t[n] + Dt
51
52
53 # Dibujamos las tres velocidades angulares
54 plt.plot(t, U[:, 0], label="omega 1")
55 plt.plot(t, U[:, 1], label="omega 2")
56 plt.plot(t, U[:, 2], label="omega 3")
57
58 plt.xlabel("tiempo")
59 plt.ylabel("velocidad angular")
60 plt.legend()
61 plt.show()
```

- **Líneas 6–8, la forma del cuerpo.** Los tres momentos de inercia. Con $I = (1, 2, 3)$ el eje intermedio es el 2 (la "hoja A4").
- **Líneas 12–18, el tiempo.** 20 000 pasos de $\Delta t = 0.001$, así que se simula hasta $t = 20$. El estado tiene 3 variables.
- **Líneas 21–30, la física.** `F(U)` es (1) tal cual: recibe $(\omega_1, \omega_2, \omega_3)$ y devuelve cómo cambia cada una.
- **Línea 34, el cuaderno de bitácora.** Tabla de $N+1$ filas (instantes) por 3 columnas $(\omega_1, \omega_2, \omega_3)$.
- **Línea 38, el lanzamiento.** Giro de velocidad 1 alrededor de e_2, torcido un 1 % hacia e_1 y e_3: $\varepsilon = 0.01$, $\Omega = 1$.
- **Líneas 42–43, el método. Euler explícito**, igual que en `Problema_1.py` del Milestone 1: siguiente = actual + $\Delta t \cdot$F(actual).
- **Líneas 47–50, el reloj.** Construye los instantes sumando Δt cada vez. Equivale a `t = np.arange(N+1)*Dt`.
- **Líneas 54–61, el dibujo.** Las tres velocidades angulares frente al tiempo.

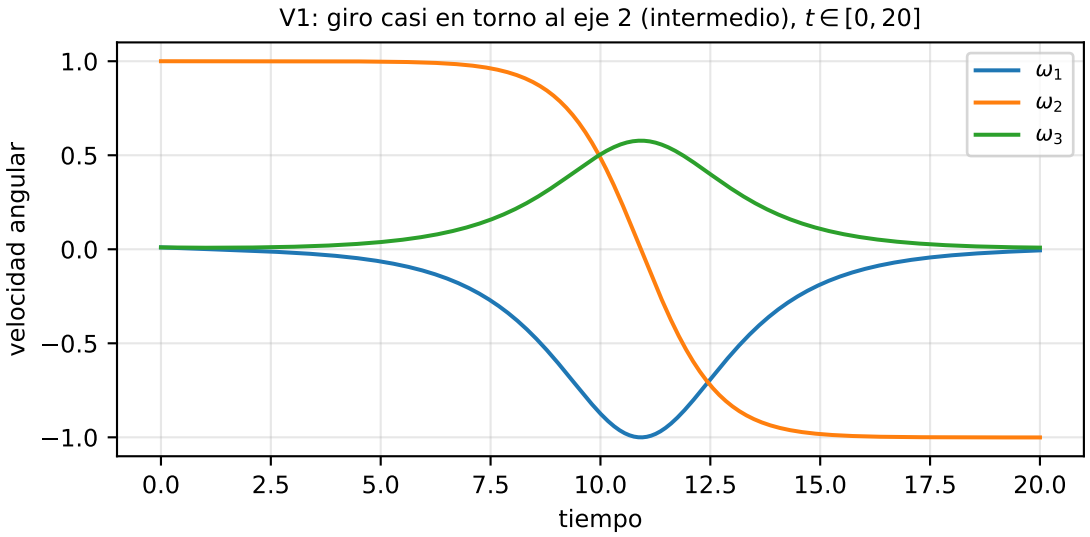


Figura 3: V1: hasta $t \approx 8$ casi no pasa nada; luego ω_2 se da la vuelta (de $+1$ a -1) mientras ω_1 y ω_3 dan un pulso. Es el primer volteo, en $t = 10.92$.

6.2. `Problema_raqueta_V2.py`: más tiempo, varios volteos

Es V1 sin comentarios y con $N = 100\,000$ pasos (hasta $t = 100$). Lo único que cambia es la línea 9:

```
1 Dt = 0.001
2 N = 100000
3 Nv = 3
```

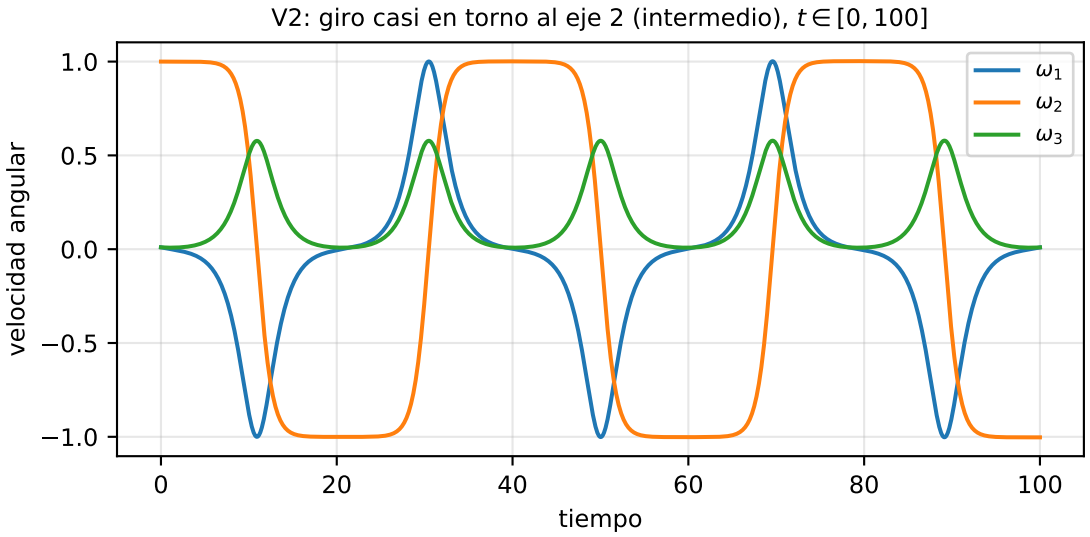


Figura 4: V2: cinco volteos regulares, cada 19.55 unidades de tiempo. Son las mesetas largas con transiciones bruscas del columpio que casi da la vuelta. ω_1 cambia de signo en cada volteo y ω_3 da siempre un pulso positivo. Es la figura de la lámina-resumen de la carpeta.

6.3. `Problema_raqueta_V3.py`: el experimento de control

Mismo código que V2, pero lanzado alrededor del eje de *menor* inercia:

```
1 # Giro casi alrededor del eje 1
2 U[0, :] = array([1.0, 0.01, 0.01])
```

Es el **experimento de control**: si el teorema es cierto, aquí no debería haber volteos.

6.4. `Problema_raqueta_V4.py`: ver girar el cuerpo (cuaterniones)

La idea: además de *cómo gira*, saber *hacia dónde apunta*

V1–V3 calculan las velocidades de giro, pero no la **orientación**: dónde está cada eje del cuerpo en cada instante. Para animarlo hace falta guardarla, y V4 lo hace con un **cuaternión**.

Analogía: una flecha y un ángulo en un paquete

Cualquier orientación se consigue girando desde la posición inicial un cierto ángulo ϕ alrededor de un cierto eje $\mathbf{n}$ (una flecha). Un cuaternión empaqueta "flecha + ángulo" en cuatro números:

$$q = \left(\cos\tfrac{\phi}{2},\ \sin\tfrac{\phi}{2}\,n_x,\ \sin\tfrac{\phi}{2}\,n_y,\ \sin\tfrac{\phi}{2}\,n_z\right), \qquad |q| = 1.$$

$q = (1, 0, 0, 0)$ significa "sin girar". Se usan en vez de ángulos de Euler (alabeo, cabeceo, guiñada) porque los ángulos de Euler se "atascan" en ciertas posturas (el *bloqueo del cardán*), y los cuaterniones no.

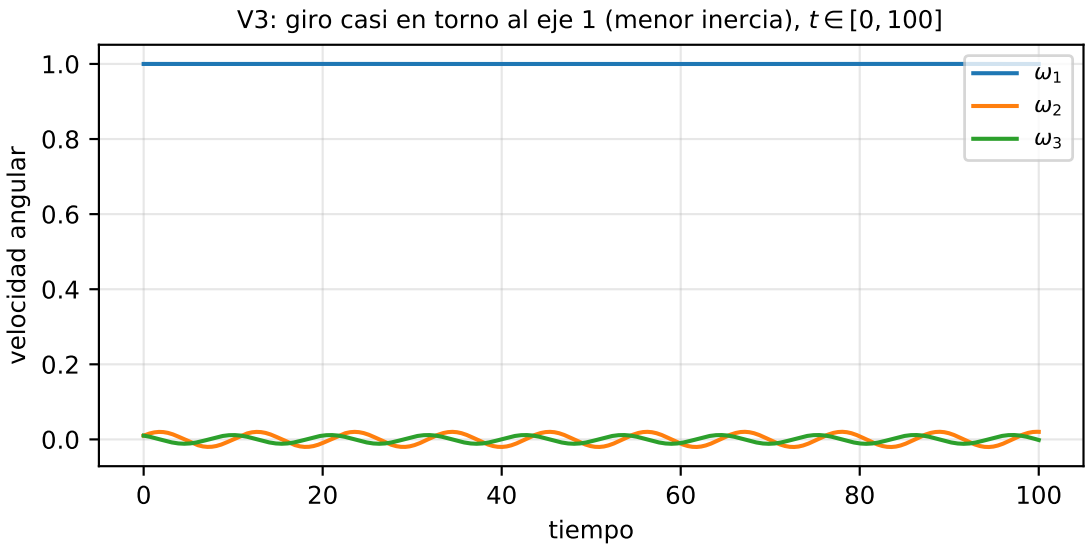


Figura 5: V3: ω_1 se queda en 1 para siempre y ω_2, ω_3 oscilan diminutas (por debajo de 0.02) con periodo 10.88, el predicho. Es el lápiz colgado: estable.

Cómo cambia la orientación. Con la velocidad angular en ejes del cuerpo, la orientación evoluciona según

$$\dot{q} = \tfrac{1}{2}\, q \otimes (0, \omega_1, \omega_2, \omega_3),$$

donde $\otimes$ es la multiplicación de cuaterniones. Para dibujar, cada cuaternión se convierte en una **matriz de rotación** $R(q)$ que lleva un vector de los ejes del cuerpo a los ejes fijos del espacio.

El código

```
def multiplicar_cuaterniones(q, p):

    q0, q1, q2, q3 = q
    p0, p1, p2, p3 = p

    return array([
        q0*p0 - q1*p1 - q2*p2 - q3*p3,
        q0*p1 + q1*p0 + q2*p3 - q3*p2,
        q0*p2 - q1*p3 + q2*p0 + q3*p1,
        q0*p3 + q1*p2 - q2*p1 + q3*p0
    ])


def matriz_rotacion(q):

    q0, q1, q2, q3 = q

    return array([
        [
            1 - 2*(q2*q2 + q3*q3),
            2*(q1*q2 - q0*q3),
            2*(q1*q3 + q0*q2)
        ],
        [
            2*(q1*q2 + q0*q3),
            1 - 2*(q1*q1 + q3*q3),
            2*(q2*q3 - q0*q1)
        ],
        [
```

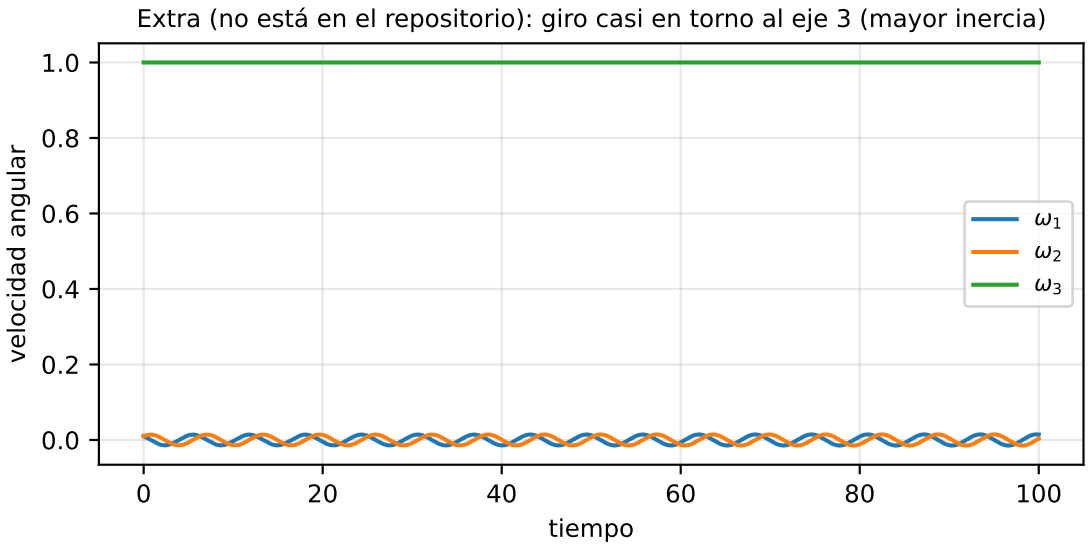


Figura 6: Extra (no está en tu código, lo he añadido para completar): lanzado alrededor del eje de *mayor* inercia, e_3. También estable, con periodo $2\pi = 6.28$, el predicho.

```
            2*(q1*q3 - q0*q2),
            2*(q2*q3 + q0*q1),
            1 - 2*(q1*q1 + q2*q2)
        ]
    ])
```

- **Líneas 44–54:** el producto de cuaterniones (producto de Hamilton), escrito componente a componente.
- **Líneas 57–77:** la fórmula estándar de la matriz de rotación a partir de q.

```
U = zeros((N + 1, 3))

# Q guarda la orientación del cuerpo
Q = zeros((N + 1, 4))


# Giro casi alrededor del eje intermedio
U[0, :] = array([0.01, 1.0, 0.01])

# Orientación inicial
Q[0, :] = array([1.0, 0.0, 0.0, 0.0])


# ========================================
# 6. INTEGRACIÓN TEMPORAL
# ========================================

for n in range(0, N):

    # Velocidad angular: Euler explícito
    U[n + 1, :] = U[n, :] + Dt * F(U[n, :])

    # Velocidad angular en forma de cuaternión
    omega = array([
        0.0,
        U[n, 0],
        U[n, 1],
```

```
        U[n, 2]
    ])

    # Cómo cambia la orientación
    dQ = 0.5 * multiplicar_cuaterniones(Q[n, :], omega)

    # Euler para la orientación
    Q[n + 1, :] = Q[n, :] + Dt * dQ

    # Normalizamos el cuaternión
    norma = sqrt((Q[n + 1, :]**2).sum())

    Q[n + 1, :] = Q[n + 1, :] / norma
```

- **Líneas 84–94:** dos cuadernos: `U` para $\boldsymbol{\omega}$ y `Q` para la orientación, que empieza sin girar, $q = (1, 0, 0, 0)$.
- **Línea 104:** Euler explícito para $\boldsymbol{\omega}$, igual que V1–V3.
- **Líneas 107–118:** $\boldsymbol{\omega}$ se mete en un cuaternión $(0, \omega_1, \omega_2, \omega_3)$, se calcula $\dot{q}$ y se avanza también con Euler explícito.
- **Líneas 121–123, la normalización:** se divide q por su tamaño para que vuelva a medir 1.

Conexión con los Milestones: por qué hay que normalizar (Pitágoras otra vez)

La ecuación $\dot{q} = \frac{1}{2} q \otimes \boldsymbol{\omega}$ conserva $|q| = 1$, igual que el oscilador conserva el radio. Euler explícito **rompe esa conservación por el mismo motivo que en el Milestone 1**: avanza en línea recta por la tangente, y por Pitágoras el tamaño crece en un factor $\sqrt{1 + \Delta t^2 |\boldsymbol{\omega}|^2 / 4}$ en cada paso. He quitado la normalización para comprobarlo: en $t = 100$ el cuaternión mide 1.0133 en lugar de 1, lo cual deformaría el cuerpo dibujado. Normalizar es "devolver el punto a la esfera" en cada paso. Arregla el tamaño, pero no el error de dirección (la fase): la orientación de V4 se desvía de la de referencia 0.3° en $t = 20$, 2° en $t = 50$ y 8.3° en $t = 100$.

```
# Dimensiones del cuerpo
a = 3.0
b = 1.2
c = 0.25

vertices = array([
    [-a/2, -b/2, -c/2],
    [ a/2, -b/2, -c/2],
    [ a/2,  b/2, -c/2],
    [-a/2,  b/2, -c/2],

    [-a/2, -b/2,  c/2],
    [ a/2, -b/2,  c/2],
    [ a/2,  b/2,  c/2],
    [-a/2,  b/2,  c/2]
])


aristas = [
    (0, 1), (1, 2), (2, 3), (3, 0),
    (4, 5), (5, 6), (6, 7), (7, 4),
    (0, 4), (1, 5), (2, 6), (3, 7)
]
```

- **Líneas 131–145:** un bloque de $3 \times 1.2 \times 0.25$ definido por sus 8 esquinas, en los ejes del cuerpo.

- **Líneas 148–152:** las 12 aristas, como pares de esquinas que se unen con una línea.

Detalle a mejorar en el código: el bloque dibujado no es el simulado

Un bloque de $3 \times 1.2 \times 0.25$ tiene momentos de inercia en proporción $1.50 : 9.06 : 10.44$, no $1 : 2 : 3$. El orden (e_2 intermedio) es el mismo, así que el dibujo es cualitativamente correcto, pero un bloque así se voltearía a otro ritmo ($\sigma = 0.82$ en vez de 0.58). Para que fueran coherentes, se podrían calcular `I1, I2, I3` a partir de `a, b, c` con las fórmulas de la sección 1.2, o dibujar una lámina de proporción $\sqrt{2}$ (la hoja A4).

```
salto = 200


def actualizar(frame):

    n = frame * salto

    R = matriz_rotacion(Q[n, :])

    # Giramos todos los vértices
    vertices_rotados = vertices @ R.T

    # Actualizamos el cuerpo
    for linea, arista in zip(lineas, aristas):

        i, j = arista

        x = [
            vertices_rotados[i, 0],
            vertices_rotados[j, 0]
        ]

        y = [
            vertices_rotados[i, 1],
            vertices_rotados[j, 1]
        ]

        z = [
            vertices_rotados[i, 2],
            vertices_rotados[j, 2]
        ]

        linea.set_data_3d(x, y, z)


    # Ejes propios del cuerpo
    longitud = 1.5

    for i in range(3):

        eje = R[:, i] * longitud

        ejes[i].set_data_3d(
            [0, eje[0]],
            [0, eje[1]],
            [0, eje[2]]
        )


    tiempo = n * Dt

    ax.set_title(f"t = {tiempo:.2f}")

```

```
    return lineas + ejes


numero_frames = N // salto


animacion = FuncAnimation(
    fig,
    actualizar,
    frames=numero_frames,
    interval=30,
    blit=False,
    repeat=True
)
```

- **Línea 194:** se dibuja un fotograma cada 200 pasos, es decir, cada 0.2 unidades de tiempo, que son 500 fotogramas en total.
- **Líneas 201–204:** se gira el bloque. `vertices @ R.T` aplica R a las 8 esquinas de golpe, porque cada fila es una esquina.
- **Líneas 207–226:** se mueven las 12 líneas de las aristas a su nueva posición.
- **Líneas 230–240:** los tres ejes del cuerpo. La columna i de R es el eje e_i visto desde el espacio.
- **Líneas 253–260:** `FuncAnimation` llama a `actualizar` para cada fotograma, cada 30 ms.

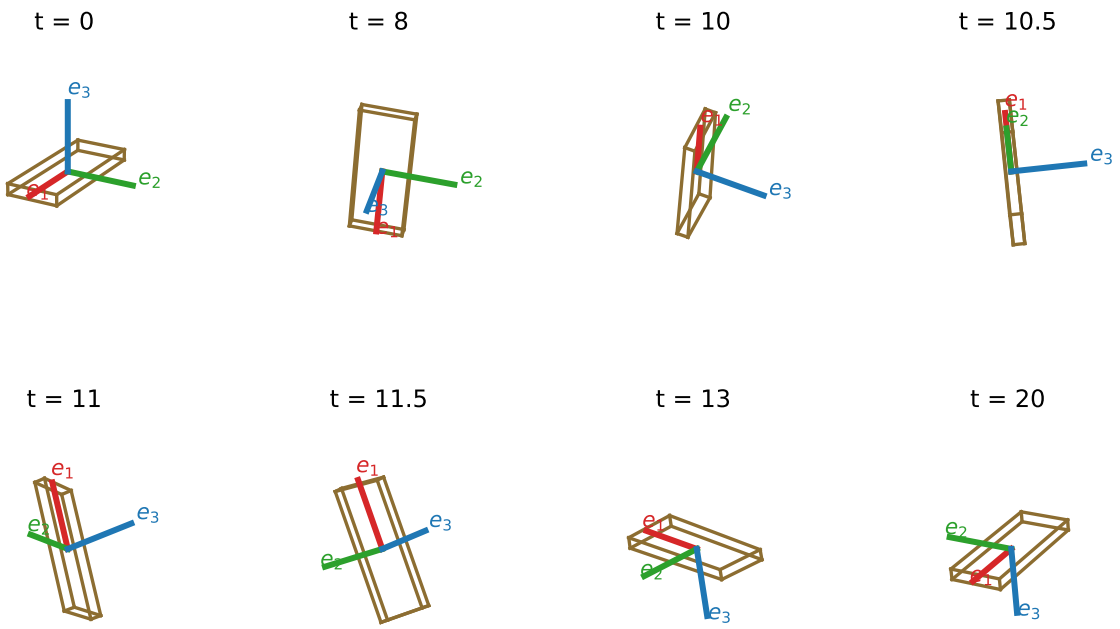


Figura 7: Fotogramas de la animación de V4 (rojo e_1, verde e_2, azul e_3). Entre $t = 0$ y $t = 8$ el bloque gira tranquilo alrededor de e_2. Entre $t \approx 10$ y $t \approx 12$ se da la vuelta: en $t = 13$ y $t = 20$, e_2 apunta al lado contrario que al principio. Es el volteo de la raqueta.

6.5. `Problema_raqueta_V5.py`: el laboratorio interactivo

```
Dt = 0.002
Tfinal = 100.0

N = int(Tfinal / Dt)


def simular(I1, I2, I3, w10, w20, w30):

    # Tabla de estados:
    # columna 0 -> omega 1
    # columna 1 -> omega 2
    # columna 2 -> omega 3

    U = zeros((N + 1, 3))

    # Condiciones iniciales

    U[0, :] = array([w10, w20, w30])


    # Función F(U)

    def F(U):

        w1 = U[0]
        w2 = U[1]
        w3 = U[2]

        dw1 = ((I2 - I3) / I1) * w2 * w3
        dw2 = ((I3 - I1) / I2) * w3 * w1
        dw3 = ((I1 - I2) / I3) * w1 * w2

        return array([dw1, dw2, dw3])


    # Euler explícito

    for n in range(0, N):

        U[n + 1, :] = U[n, :] + Dt * F(U[n, :])


    # Vector de tiempos

    t = zeros(N + 1)

    for n in range(0, N):

        t[n + 1] = t[n] + Dt


    return t, U
```

- **Líneas 10–13:** ahora se fija el tiempo final (100) y el paso ($\Delta t = 0.002$, el doble que antes), y se calcula $N = 50\,000$.
- **Líneas 16–61, `simular`:** empaqueta toda la simulación en una función que recibe las inercias y el giro inicial y devuelve $(t, \mathbf{U})$. `F` se define *dentro*, así que "recuerda" los I que se le han pasado. Es el mismo truco de la clausura que `CN` usaba en el Milestone 2.

```
def actualizar(event):

```

```
    # Leer los valores actuales de los sliders

    I1 = slider_I1.val
    I2 = slider_I2.val
    I3 = slider_I3.val

    w10 = slider_w1.val
    w20 = slider_w2.val
    w30 = slider_w3.val


    # Nueva simulación

    t, U = simular(
        I1,
        I2,
        I3,
        w10,
        w20,
        w30
    )


    # Cambiar los datos de las curvas

    linea1.set_ydata(U[:, 0])
    linea2.set_ydata(U[:, 1])
    linea3.set_ydata(U[:, 2])


    # ------------------------------------------------------
    # Descubrir cuál es el eje intermedio
    # ------------------------------------------------------

    inercias = [
        (I1, "eje 1"),
        (I2, "eje 2"),
        (I3, "eje 3")
    ]

    inercias_ordenadas = sorted(inercias)

    eje_intermedio = inercias_ordenadas[1][1]


    # ------------------------------------------------------
    # Descubrir alrededor de qué eje estamos girando más
    # ------------------------------------------------------

    velocidades = [
        abs(w10),
        abs(w20),
        abs(w30)
    ]

    indice_dominante = velocidades.index(
        max(velocidades)
    )

    eje_dominante = indice_dominante + 1


    # Texto de diagnóstico

```

```
    mensaje = (
        f"Eje intermedio: {eje_intermedio}\n"
        f"Giro inicial principal: eje {eje_dominante}"
    )

    texto_info.set_text(mensaje)


    # Redibujar

    fig.canvas.draw_idle()
```

- **Líneas 238–256:** lee los seis deslizadores (tres inercias y tres velocidades iniciales) y vuelve a simular.
- **Líneas 261–263:** cambia los datos de las curvas sin crear una figura nueva.
- **Líneas 270–278:** averigua cuál es el eje intermedio ordenando las inercias de menor a mayor y quedándose con la del medio.
- **Líneas 285–295:** averigua alrededor de qué eje se ha lanzado, que es el de la $|\omega|$ inicial más grande.
- **Líneas 300–305:** escribe el diagnóstico. Si los dos ejes coinciden, el teorema predice volteos.

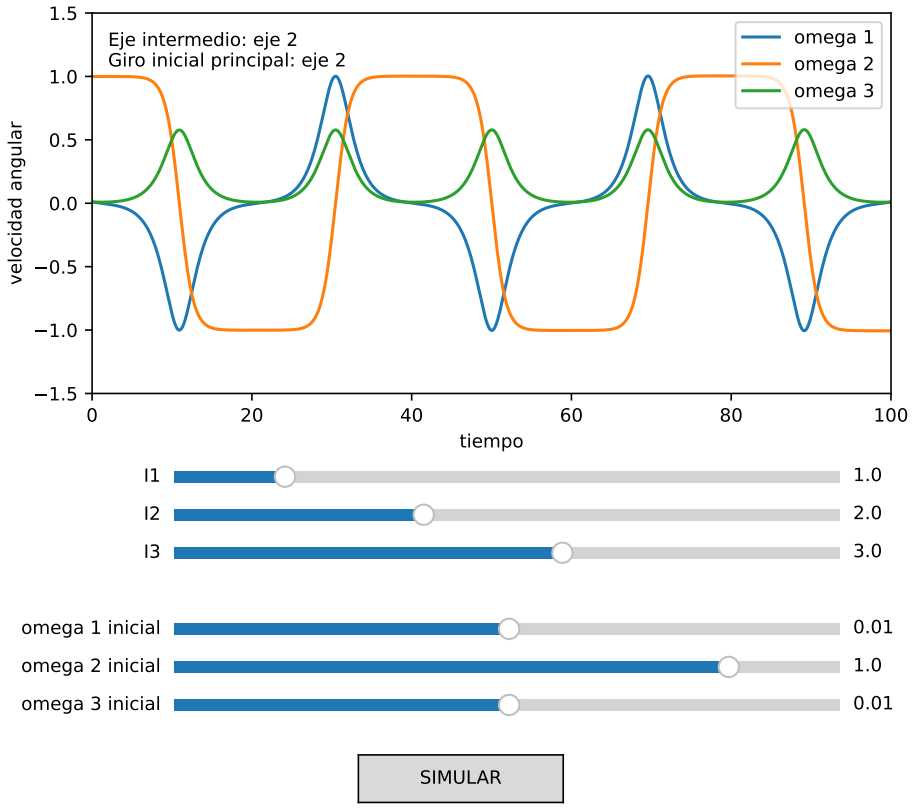


Figura 8: La ventana de V5 con los valores iniciales. Arriba a la izquierda aparece el diagnóstico: eje intermedio el 2 y giro principal alrededor del 2, así que hay volteos.

Detalle a mejorar en el códigos

- **Inercias imposibles:** los deslizadores permiten, por ejemplo, $I = (0.2, 0.2, 5)$, que no corresponde a ningún cuerpo real (debe cumplirse $I_i \leq I_j + I_k$). Se podría avisar en el texto.
- **Inercias iguales:** si dos inercias coinciden no hay eje intermedio (el cuerpo es simétrico, como una peonza), pero `sorted` elegirá uno igualmente y el diagnóstico dirá un eje intermedio que no existe.
- **Eje vertical fijo** en ± 1.5 (línea 128): con velocidades iniciales más grandes, las curvas se salen de la gráfica.
- **Euler explícito con $\Delta t = 0.002$:** la energía crece un 1.2 % hasta $t = 100$, el doble que en V2 (siguiente sección).

7. ¿Es fiable la simulación? (conexión con los Milestones)

7.1. Euler explícito siempre añade energía

Igual que en el oscilador, Euler explícito **hace crecer** la energía y el momento angular, y siempre hacia arriba. El motivo es el mismo Pitágoras:

$$2T_{n+1} = 2T_n + \underbrace{2\Delta t \sum_i I_i \omega_i F_i}_{=0 \text{ (la física conserva la energía)}} + \underbrace{\Delta t^2 \sum_i I_i F_i^2}_{\geq 0}.$$

El paso va "perpendicular" a la dirección en que cambiaría la energía, así que solo queda el término al cuadrado, que es positivo. Lo mismo ocurre con $|\mathbf{L}|^2$.

Lo que dice la simulación

- Con $\Delta t = 0.001$ (V2), la energía crece un 0.58 % hasta $t = 100$. Con $\Delta t = 0.002$ (V5), un 1.16 %: el doble, como corresponde a un método de orden 1.
- **El error se comete en los volteos** (Fig. 9, abajo): la energía sube a escalones. En las mesetas $\omega_1 \approx \omega_3 \approx 0$, así que $\mathbf{F} \approx \mathbf{0}$ y el término $\Delta t^2 \sum I_i F_i^2$ casi no suma nada. Durante el volteo $\mathbf{F}$ es grande y ahí se acumula el error.
- Aun así, **los volteos caen en el mismo instante** que en la solución de referencia (diferencias de 0.01). El error en la energía cambia muy poco la órbita, y el periodo depende de ella de forma suave.
- RK4 con el mismo paso conserva la energía hasta 2×10^{-14}, el límite de la máquina.

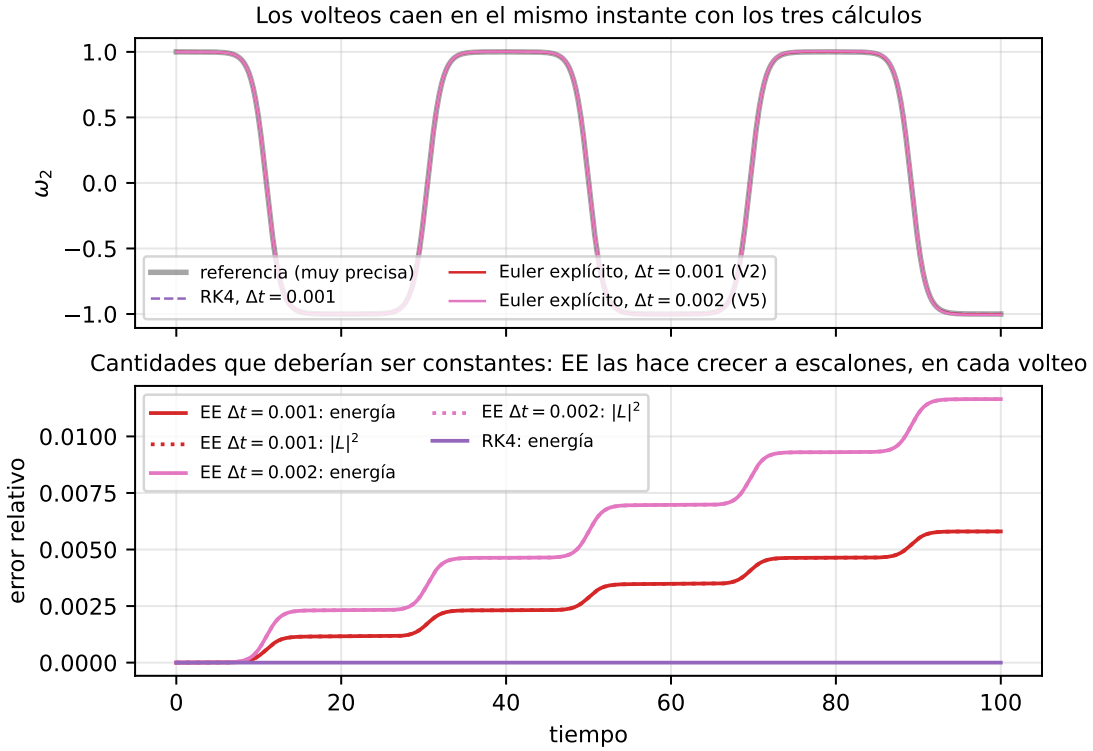


Figura 9: Arriba: ω_2 con Euler explícito (V2 y V5), RK4 y una referencia muy precisa; las cuatro curvas se superponen. Abajo: error relativo de la energía y de $|\mathbf{L}|^2$. Euler explícito sube a escalones, uno por volteo, y RK4 se queda en cero.

7.2. Con los métodos del Milestone 3

Como `F` tiene la forma de un problema de Cauchy, se puede pasar directamente a `integrar_Cauchy` del Milestone 3 con `F(U, t)`. Lo he hecho con un paso grande, $\Delta t = 0.05$, para que las diferencias se vean. También he añadido el **punto medio implícito**, que no está en el repositorio.

Lo que dice la simulación

Error relativo máximo en la energía hasta $t = 100$ con $\Delta t = 0.05$:

Método	Error en la energía	Comentario
EE	3.7×10^{-1} (37 %)	la órbita se infla visiblemente
CN (trapecio)	2.1×10^{-4}	pequeño, pero **no cero**
RK4	6.9×10^{-9}	excelente
Punto medio implícito	3.8×10^{-15}	**exacto** (redondeo de la máquina)

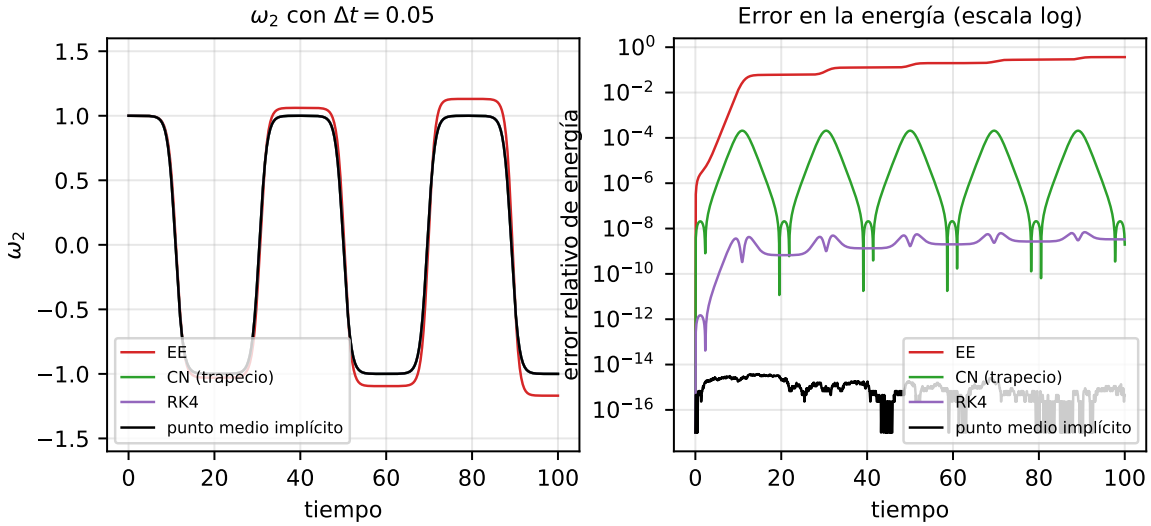


Figura 10: Los métodos del Milestone 3 aplicados a la raqueta con $\Delta t = 0.05$. Izquierda: con EE los volteos llegan a ± 1.15 y no a ± 1 (la energía se ha inflado). Derecha: error en la energía, en escala logarítmica.

Idea clave: CN y el punto medio no son lo mismo aquí

En el oscilador, CN conservaba la energía exactamente, y CN y el punto medio implícito daban lo mismo. Aquí CN **no** conserva exactamente la energía, y el punto medio sí. La razón:

- El oscilador es *lineal*, y con un $\mathbf{F}$ lineal "media de las direcciones" (CN) y "dirección en el punto medio" coinciden.
- Las ecuaciones de Euler son *no lineales* (productos $\omega_i \omega_j$), y ahí CN y el punto medio son métodos distintos.
- El punto medio implícito tiene una propiedad especial: conserva *exactamente* todas las cantidades cuadráticas del problema, y la energía y $|\mathbf{L}|^2$ lo son. CN no tiene esa propiedad cuando $\mathbf{F}$ no es lineal.

Es lo que se avisaba en el Milestone 1 sobre el nombre de `Problema_2_2.py`: para el oscilador daba igual llamarlo "punto medio" o "trapecio", pero en problemas no lineales no.

7.3. Recomendación

Para este problema, Euler explícito con $\Delta t = 0.001$ da una imagen *cualitativamente* correcta: volteos en el instante correcto y error de energía por debajo del 1 %. Para resultados de calidad sin reducir el paso, lo natural es reutilizar lo hecho en los Milestones: `integrar_Cauchy("RK4", ...)` del Milestone 3, o un punto medio implícito si importa conservar la energía exactamente durante mucho tiempo.

8. Resumen

Eje	Inercia	Perturbación	Analogía	En la esfera	Tu simulación
e_1	menor	oscila (periodo 10.88)	lápiz colgado	cima, circulitos	V3: estable
e_2	**intermedio**	**crece** ($e^{0.577t}$)	lápiz en la punta	puerto, cruce en X	V1, V2, V4: volteo cada 19.55
e_3	mayor	oscila (periodo 6.28)	lápiz colgado	valle, circulitos	extra: estable

- **Por qué ocurre:** alrededor del eje intermedio, las dos perturbaciones se alimentan una a otra (el producto ab es positivo) y crecen exponencialmente.
- **Por qué vuelve:** la energía y el momento angular se conservan y obligan al movimiento a recorrer una curva cerrada. Los volteos son periódicos, no caóticos.
- **Por qué es tan fácil de ver:** el tiempo hasta el primer volteo solo crece con el logaritmo de lo perfecto que sea el lanzamiento.
- **Numéricamente:** Euler explícito infla la energía (Pitágoras, como en el oscilador) sobre todo durante los volteos. RK4 y el punto medio implícito lo resuelven.

Apéndice: observaciones sobre el código

1. Todas las versiones usan Euler explícito. La energía crece un 0.58 % (V2–V4, $\Delta t = 0.001$) o un 1.16 % (V5, $\Delta t = 0.002$) hasta $t = 100$. Con RK4 del Milestone 3 desaparecería.
2. El código de integración está copiado en V1, V2, V3 y V5. Se podría escribir `F(U, t)` una vez y usar `integrar_Cauchy` del Milestone 3.
3. `Problema_raqueta_V4.py`: el bloque dibujado ($3 \times 1.2 \times 0.25$) no tiene los momentos de inercia simulados $(1, 2, 3)$.
4. `Problema_raqueta_V5.py`: admite inercias físicamente imposibles; con dos inercias iguales el diagnóstico de "eje intermedio" no tiene sentido; el eje vertical está fijo en ± 1.5.
5. `r.py` está vacío.
6. Menores: el vector de tiempos se construye con un bucle (equivale a `np.arange(N+1)*Dt`).